

Tree Methods II

Sean Hellingman ©

Multivariate Statistics for Applied Data Science (ADSC 4050)

shellingman@tru.ca

Winter 2026



THOMPSON RIVERS UNIVERSITY

Topics

- 2 Recap
- 3 Introduction
- 4 Regression Trees
- 5 Predictions
- 6 Coming Up
- 7 Exercises and References

Recap

- Last time we covered **Tree Methods I**:
 - Classification trees.
 - Procedure.
 - Interpreting results.

Introduction

- The same approach to classification trees may be used in regression problems.
 - Continuous response variable.
- The dataset is divided via binary splits into groups such that the observations in each node are as similar as possible.
 - Divided so that the variance is minimized in each group.
- *Non-parametric approach that does not require pre-processing.*

Overview

- **Tree-based** methods partition the feature space into a number of smaller (non-overlapping) regions with similar response values using a set of splitting rules.
- There are many methods for constructing decision trees.
 - The **classification and regression tree** (CART) algorithm is the most well known.
- The same approach can be taken for both continuous and categorical response variables.
 - **Classification trees** refer to analyses that use categorical data for the response variable.
 - **Regression trees** refer to analyses that use continuous data for the response variable.

Regression Trees

Regression Trees

- **Regression trees** refer to analyses that use continuous data for the response variable.

$$SSE = \sum_{i \in R_1} (y_i - c_1)^2 + \sum_{i \in R_2} (y_i - c_2)^2 \quad (1)$$

- Procedure:
 - 1 Partition the training data into homogeneous subgroups.
 - 2 Fit a simple constant in each subgroup.
 - 3 The subgroups (nodes) are formed recursively using binary partitions formed by asking simple yes-or-no questions about each variable.
 - 4 This is done a number of times until a suitable stopping criteria is satisfied.
 - 5 The model predicts the output based on the class that has majority representation.

Illustrative Example 1

- Run the provided code and examine the two-dimensional regression tree.
- What did you notice?

Regression Trees in R

- We can use the *rpart* and the *rpart.plot* packages in R.
 - In Python
- Usage:
 - `tree <- rpart(response ~ X1 + X2 + ... , data = df, method = "anova")`
 - `rpart.plot(tree)`
- Note:
 - `method = "poisson"`: If *y* has 2 columns
 - `method = "exp"`: If *y* is a survival object
 - `method = "class"`: If *y* is a factor

Example 1

- Load the *Wages.csv* dataset and do the following:
 - ① Construct a regression tree with *Salary* as the response variable (ignore: *ID* and *Salary.5*).
 - ② Examine your tree.
- What do you notice?

Recall: Tree Depth

- The tree can continue to *grow* until each object is *correctly* grouped.
 - This will lead to overfitting and poor generalization performance.
- There is a balance that needs to be found between complexity and generalization performance.
- Two primary ways this can be done:
 - ① **Early stopping:** Restrict the tree depth to a certain level or to restrict the minimum number of observations allowed in any terminal node.
 - ② **Pruning:** Grow a very large, complex tree and then prune it back to find an optimal subtree.

Early Stopping

- **Early stopping** explicitly restricts the growth of the tree.
- The most common approaches are to restrict the tree depth to a certain level or to restrict the minimum number of observations allowed in any terminal node.
- To apply these methods in R:
 - Depth restriction: `maxdepth = k`
 - Minimum in leaf node: `minbucket = 1`

Example 2

- Load the *Wages.csv* dataset and do the following:
 - 1 Construct a regression tree with Salary as the response variable (ignore: ID and Salary.5).
 - 2 Examine your tree.
 - 3 Adjust the maximum depth of your tree
 - 4 Adjust the minimum number of observations allowed in the leaf nodes.
 - 5 Compare the MSE and MAE values on the training sets for your different trees.
- What do you notice?

Pruning

- **Pruning** is the process of growing a very large, complex tree and then *pruning* it back to find an optimal subtree.
- We can use the following minimization function:

$$\text{minimize}\{SSE + \alpha|T|\} \quad (2)$$

- Similar to regularization problems, the size of α will determine the complexity of the tree.
- Evaluate multiple models across a spectrum of α and use CV to identify the optimal value.

Pruning in R

- The `cp` (complexity parameter) helps to determine the complexity of the trees in R.
 - *Smaller values indicate more complex trees.*
- Construct a complex tree and use cross validation to identify the best value for α :
 - `plotcp(tree)`
 - `tree$cpstable[which.min(tree$cpstable[, "xerror"]), "CP"]`
 - `bestcp <-`
 `tree$cpstable[which.min(tree$cpstable[, "xerror"]), "CP"]`
 - `tree.pruned <- prune(tree, cp = bestcp)`

Example 3

- Load the *Wages.csv* dataset and do the following:
 - 1 Construct a very deep regression tree with Salary as the response variable (ignore: ID and Salary.5).
 - 2 Examine your tree.
 - 3 Adjust the provided code to prune your complex tree.
 - 4 Examine your new tree.
 - 5 Compare the MSE and MAE values on the training sets for your different trees.
- What do you notice?

Regression Tree Comments

- Regression trees provide the basis of more robust procedures and are very similar to classification trees.
- The complexity of the tree is related to your objective.
 - Individual trees generally do not possess sophisticated predictive capabilities.
- Trees require very little pre-processing!
 - We did not transform any of our variables in these examples.
- You can use this approach for Poisson and survival models.

Example 4

- Load the *Basketball.csv* data and do the following:
 - ① Construct a tree for a points-per-game count variable.
 - ② Adjust the explanatory variables to see if you can find something informative.
 - ③ What happens when you change the response to points-per-minute?
 - ④ Plot your trees.
- What do you notice?

Predictions

Predictions with Tree Methods

- Tree methods form the base for more robust prediction algorithms.
- The trees themselves can be used as predictive models on their own.
 - *Simple to construct and to predict with.*
- *As always, proper cross-validation should be conducted before applying to a testing set.*

Predictions in R

- To make predictions from trees in R:
 - Classification tree: `predict(tree, new.data, type = "class")`
 - Regression tree: `predict(tree, new.data)`
- To assess accuracy:
 - Classification tree: `confusionMatrix()`
 - Regression tree: `mse()`, `rmse()`, `mae()`, ...

Example 5

- Load the *ptitanic* data from the *rpart* package and do the following:
 - 1 Set your seed and split your data into a train and test set.
 - 2 Construct a very deep binary classification tree on your training set with `survived` as the response variable.
 - 3 Examine your tree.
 - 4 Adjust the provided code to prune your complex tree.
 - 5 Examine your new tree.
 - 6 Construct a confusion matrix for each of your trees.
- What do you notice?

Example 6

- Load the *Wages.csv* dataset and do the following:
 - 1 Set your seed and split your data into a train and test set.
 - 2 Construct a very deep regression tree with `Salary` as the response variable (ignore: `ID` and `Salary.5`) on your training set.
 - 3 Examine your tree.
 - 4 Adjust the provided code to prune your complex tree.
 - 5 Examine your new tree.
 - 6 Compare the MSE and MAE values on the **testing sets** for your different trees.
- What do you notice?

Tree Method Comments

- We will use these concepts in more robust prediction algorithms.
- They are *easy* non-parametric approaches to classification and regression problems.
- The binary nature of the splits make inferences less informative.

Next Lecture

- We will discuss **Bagging**.
- Review of bootstrapping.
- Introduction to ensemble methods.

Exercise 1

- Import the wine dataset from the *HDclassif* package.
- Take some time to get to know the data (including visualizations).
- Create a 75-25 train/test split.
- Construct an out-of-the box classification tree for your training dataset.
- Use early stopping to construct simpler classification trees for this dataset.
- Find the ideal complexity parameter (cp) using cross validation and construct this tree.
- Finally, compare the classification accuracy of all of your trees on the

Exercise 2

- Import the diamonds dataset from the *ggplot2* package.
- Take some time to get to know the data (including visualizations).
- Create a 75-25 train/test split.
- Construct an out-of-the box regression tree for the price of the diamonds in your training dataset.
- Use early stopping to construct simpler regression trees for this dataset.
- Find the ideal complexity parameter (*cp*) using cross validation and construct this tree.
- Finally, compare the accuracy of all of your trees on the testing set.

Exercise 3

- Import the diabetic dataset from the the *survival* package.
- Use the methods that we have covered to construct an appropriate survival regression tree for these data.
- Plot your tree.
- What do you notice?

References & Resources

- 1 Bakker, J. D. (2026) Applied Multivariate Statistics in R
 - 2 Peng R. D. (2020) *Exploratory Data Analysis with R*
 - 3 Timbers T., Campbell T., and Lee M. (2024) *Data science: A first introduction*. Chapman and Hall/CRC
 - 4 Bradley Boehmke B., and Greenwell B. (2020) *Hands-On Machine Learning with R*. Taylor & Francis
- Trees & Classification
 - CART
 - `rpart()`
 - `rpart.plot()`
 - `vip()`
 - `confusionMatrix()`